NSYSU - Data Structure – Fall 2026

Homework 3

Design: Maze Solver Design Using Depth-First Search with Stack

Data Preparation

For this assignment, you will be provided with a zip file named `HW3.zip`, which contains template files and public test data. Your primary objective is to implement the `search_from_stack()` function in the provided template file, using either Python or C++. Below is an overview of the directory structure and the contents in the zip file:

1. Python Implementation (`Py/` directory):
   - ✓ `maze_stack.py`: This is where you will implement your `search_from_stack()` function.
   - ✓ `maze.py`: Contains a reference implementation of the search algorithm using recursion, for your review and understanding.
   - ✓ `*.txt`: Includes test data files for mazes and paths.
2. C++ Implementation (`Cpp/` directory):
   - ✓ `maze_stack.cpp:` This is where you will implement your `search_from_stack()` function.
   - ✓ `maze.cpp:` Provides a reference implementation using recursion, which can help guide your own implementation.
   - ✓ `*.txt:` Test data files for mazes and paths.
   - ✓ `CTurtle.hpp`, `CImg.h`, `maze_sprites.hpp`: Headers for the graphical display. You do not need to modify them.
   - ✓ `pythonds3/cppds/stack.hpp`: The course `Stack<T>` header, the same file used in the lectures and the C++ notebook kernel.

Description

In our recent lessons, we've explored how recursive implementations can be transformed into stack-based approaches. This assignment requires you to apply this concept by designing a maze solver that employs depth-first search (DFS), similar to the examples in our textbook. Unlike the examples, however, you are tasked with utilizing a stack-based approach. The assignment is structured into three main parts as follows:

1. Modify the maze search algorithm in `maze.py` (for Python) or `maze.cpp` (for C++) so that the calls to `search_from()` follow a different order. After making these adjustments, run the program to observe any resultant changes in behavior.

Explain the significance of call order in the algorithm's behavior and explain whether it affects the algorithm's ability to find a path through the maze.

2. Function Implementation:
   - ✓ Implement the function named `search_from_stack()` in the provided template file. For Python, use `maze_stack.py`. For C++, use `maze_stack.cpp`.
3. Discussion:
   - ✓ Discuss the difference between recursion and stack implementation and their pros and cons.
   - ✓ Trace the execution of both implementations on a small maze to illustrate how they explore the maze and handle backtracking. See the Report section for detailed requirements.

Specifications

1. Function name: `search_from_stack()`
2. Input: The function takes three parameters: `maze`, which is a custom class we define representing the maze, and `start_row, start_col,` which are the starting coordinates.
3. Output: The function should return a Boolean value to indicate whether a path has been found. It should also return the found path from start to exit as a list or vector of tuples/pairs, representing the row and column coordinates.
4. **The `maze` is stored as a matrix, detailed in our textbook.**
5. **Use the `list` (in Python) or a `vector` (in C++) to store the found path. The path should be a list of tuples that store the row and column coordinates as tuples. In C++, you should use `vector` of pairs. (e.g. [(3,4), (3,5)….])**
6. The depth-first search algorithm should be used for this implementation, following the same order of the original recursion implementation.
7. You are allowed to use only the standard libraries of Python or C++. In addition, use the `Stack`() class from our provided code base. For C++, include `pythonds3/cppds/stack.hpp` (already in the `Cpp/` folder; it is the same header used in the lectures) and use `Stack<T>` with `push()`, `pop()`, `peek()`, `isEmpty()` and `size()`. You do not need to download anything else. An online copy of the course headers is at github.com/phonchi/pythonds3/tree/master/cppds; ignore its `deprecated/` folder, which keeps old 2024 code (such as `stack.cpp`) that the course no longer uses.
8. Do not focus on visualization for this assignment; your primary goal is to ensure the correct path and Boolean value are returned. However, if you are interested, feel free to explore the GUI code or display your path.

Usage of the programs

1. Use `python .\maze_stack.py .\maze1.txt` (or maze.py for the recursive version) to execute the program and run the maze solver with the specified maze file.
2. Use `python .\maze_stack.py .\maze1.txt .\correct_path1.txt` to compare the solution generated by your program against a correct path file, verifying the implementation's accuracy.
3. Use `python .\maze_stack.py .\maze1.txt --nogui` if you wish to run the program without the graphical user interface, suitable for environments where GUI support is unavailable or unnecessary.
4. Compile the C++ program with GNU C++17 (`-std=gnu++17`; plain `-std=c++17` hides `M_PI`, which the CTurtle library needs on Windows). Ensure your GCC compiler version is above 7.0. On Windows: `g++ -std=gnu++17 -O2 maze_stack.cpp -o maze_stack.exe -lgdi32 -luser32 -pthread`. On Linux, link `-lX11 -pthread` instead of the Windows libraries. Without a display, add `-Dcimg_display=0` and always run with `--nogui`. The course headers are found through the relative path in the `#include` lines, so compile inside the `Cpp/` folder. For compilation assistance, feel free to reach out to us.
5. Run it like the Python version: `.\maze_stack.exe .\maze1.txt .\correct_path1.txt --nogui` (the correct-path file and `--nogui` are optional).

Deliverables

1. Deadline: 2026/11/29 (Sun.), 11:59 PM. Hand in the following two items to the cyber universities.
2. Report:
   - ✓ Modify the maze search program in `maze.py` (or `maze.cpp` for C++) so that the calls to `search_from()` are in a different order. Observe the changes in the program's behavior, explain the reasons for any differences, and discuss the impact of call order on the algorithm's ability to find a path through the maze.
   - ✓ Describe the design of your program and the data structures you utilized. Discuss what you have learned from completing this homework.
   - ✓ Analyze the differences between recursion and stack implementation, including their advantages and disadvantages.
   - ✓ Choose or design a small maze that requires at least one backtracking step. Include the maze layout in your report and trace the execution of both the

recursive and stack-based DFS implementations on the same maze.

- **Recursive DFS**: Show the sequence of recursive calls and returns, including at least one backtracking step.
- **Stack-Based DFS:** Show the push and pop operations of your search_from_stack() implementation, and explain how it handles dead ends and continues searching.

You may use a table or diagram to illustrate the execution process. It is not necessary to trace the entire maze.

3. Program Source Files:
   - ✓ Submit your source files in a zip file. **Ensure that you follow the provided template files**.
   - ✓ Source File Comments: Each file must begin with three lines of comments indicating the Author, Date, and Purpose of the program. Include appropriate comments throughout your code for clarity.

Grading Policy

- Function Correctness: 60% (36% for public test cases and 24% for hidden test cases).
- Report and discussion: 40%.

Reference

1. https://runestone.academy/ns/books/published/pythonds3/Recursion/ExploringaMaze.html
2. https://medium.com/swlh/solving-mazes-with-depth-first-search-e315771317ae
3. https://varsubham.medium.com/maze-path-finding-using-dfs-e9c5fa14106f
4. https://sqlpad.io/tutorial/python-maze-solver/